aa274a/hw1/P2_rrt.py

```python
import numpy as np
import matplotlib.pyplot as plt
from utils import plot_line_segments, line_line_intersection

class RRT(object):
    """ Represents a motion planning problem to be solved using the RRT algorithm"""
    def __init__(self, statespace_lo, statespace_hi, x_init, x_goal, obstacles):
        self.statespace_lo = np.array(statespace_lo)    # state space lower bound (e.g., [-5, -5])
        self.statespace_hi = np.array(statespace_hi)    # state space upper bound (e.g., [5, 5])
        self.x_init = np.array(x_init)                  # initial state
        self.x_goal = np.array(x_goal)                  # goal state
        self.obstacles = obstacles                      # obstacle set (line segments)
        self.path = None        # the final path as a list of states

    def is_free_motion(self, obstacles, x1, x2):
        """
        Subject to the robot dynamics, returns whether a point robot moving
        along the shortest path from x1 to x2 would collide with any obstacles
        (implemented as a "black box")

        Inputs:
            obstacles: list/np.array of line segments ("walls")
            x1: start state of motion
            x2: end state of motion
        Output:
            Boolean True/False
        """
        raise NotImplementedError("is_free_motion must be overriden by a subclass of RRT")

    def find_nearest(self, V, x):
        """
        Given a list of states V and a query state x, returns the index (row)
        of V such that the steering distance (subject to robot dynamics) from
        V[i] to x is minimized

        Inputs:
            V: list/np.array of states ("samples")
            x - query state
        Output:
            Integer index of nearest point in V to x
        """
        raise NotImplementedError("find_nearest must be overriden by a subclass of RRT")

    def steer_towards(self, x1, x2, eps):
```

```
        """
        Steers from x1 towards x2 along the shortest path (subject to robot
        dynamics). Returns x2 if the length of this shortest path is less than
        eps, otherwise returns the point at distance eps along the path from
        x1 to x2.

        Inputs:
            x1: start state
            x2: target state
            eps: maximum steering distance
        Output:
            State (numpy vector) resulting from bounded steering
        """
        raise NotImplementedError("steer_towards must be overriden by a subclass of RRT")

    def solve(self, eps, max_iters=1000, goal_bias=0.05, shortcut=False, plot=True):
        """
        Constructs an RRT rooted at self.x_init with the aim of producing a
        dynamically-feasible and obstacle-free trajectory from self.x_init
        to self.x_goal.

        Inputs:
            eps: maximum steering distance
            max_iters: maximum number of RRT iterations (early termination
                is possible when a feasible solution is found)
            goal_bias: probability during each iteration of setting
                x_rand = self.x_goal (instead of uniformly randly sampling
                from the state space)
            shortcut: whether to call shortcut_path on the solution (and, if
                plot is True, also plot the path before and after shortcutting)
            plot: whether to plot the tree and solution (set False for
                batch experiments)
        Output:
            Boolean, True if a solution was found. See also the
            "Intermediate Outputs" descriptions below
        """

        state_dim = len(self.x_init)

        # V stores the states that have been added to the RRT (pre-allocated at its maximum size
        # since numpy doesn't play that well with appending/extending)
        V = np.zeros((max_iters + 1, state_dim))
        V[0,:] = self.x_init    # RRT is rooted at self.x_init
        n = 1                   # the current size of the RRT (states accessible as V[range(n),:])

        # P stores the parent of each state in the RRT. P[0] = -1 since the root has no parent,
```

```
        # P[1] = 0 since the parent of the first additional state added to the RRT must have been
        # extended from the root, in general 0 <= P[i] < i for all i < n
        P = -np.ones(max_iters + 1, dtype=int)


        success = False
        self.num_iters = 0

        ## Intermediate Outputs
        # You must update and/or populate:
        #    - V, P, n: the representation of the planning tree
        #    - success: whether or not you've found a solution within max_iters RRT iterations
        #    - self.path: if success is True, then must contain list of states (tree nodes)
        #          [x_init, ..., x_goal] such that the global trajectory made by linking steering
        #          trajectories connecting the states in order is obstacle-free.
        #    - self.num_iters: the number of RRT iterations (loop passes, including ones
        #          whose extension was rejected by collision checking) that were run before
        #          returning, i.e. the iteration at which the goal was reached, or max_iters
        #          if no solution was found.

        ## Hints:
        #   - use the helper functions find_nearest, steer_towards, and is_free_motion
        #   - remember that V and P always contain max_iters elements, but only the first n
        #     are meaningful! keep this in mind when using the helper functions!
        #   - the order in which you pass in arguments to steer_towards and is_free_motion is important

        ########## Code starts here ##########

        self.path = []

        def rand_num(high, low):
            x_range = high[0] - low[0]
            y_range = high[1] - low[1]
            return np.array([np.random.random()*x_range + low[0], np.random.random()*y_range + low[1]])

        for iter in range(max_iters):
            self.num_iters = iter

            z = np.random.random()

```

```
            if z < goal_bias:
                x_rand = self.x_goal
            else:
                x_rand = rand_num(self.statespace_hi, self.statespace_lo)

            x_near_idx = self.find_nearest(V[:n], x_rand)
            x_near = V[x_near_idx]

            x_new = self.steer_towards(x_near, x_rand, eps)

            if self.is_free_motion(self.obstacles, x_near, x_new):
                V[n] = x_new
                P[n] = x_near_idx

                n += 1

                if np.allclose(x_new, self.x_goal):
                    success = True
                    break


        else:
            success = False

        idx = n-1
        while idx != -1:
            self.path.append(V[idx])
            idx = P[idx]

        self.path = np.array(self.path[::-1])


        ########## Code ends here ##########

        if not plot:
            if success and shortcut:
                self.shortcut_path()
            return success

        plt.figure()
        self.plot_problem()
        self.plot_tree(V, P, color="blue", linewidth=.5, label="RRT tree", alpha=0.5)
        if success:
            if shortcut:
                self.plot_path(color="purple", linewidth=2, label="Original solution
path")
                self.shortcut_path()
                self.plot_path(color="green", linewidth=2, label="Shortcut solution
path")
            else:
                self.plot_path(color="green", linewidth=2, label="Solution path")
```

```
180             plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.03),
    fancybox=True, ncol=3)
181             plt.scatter(V[:n,0], V[:n,1])
182         else:
183             print("Solution not found!")
184 
185         return success
186 
187     def plot_problem(self):
188         plot_line_segments(self.obstacles, color="red", linewidth=2,
    label="obstacles")
189         plt.scatter([self.x_init[0], self.x_goal[0]], [self.x_init[1],
    self.x_goal[1]], color="green", s=30, zorder=10)
190         plt.annotate(r"$x_{init}$", self.x_init[:2] + [.2, 0], fontsize=16)
191         plt.annotate(r"$x_{goal}$", self.x_goal[:2] + [.2, 0], fontsize=16)
192         plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.03), fancybox=True,
    ncol=3)
193         plt.axis('scaled')
194 
195     def shortcut_path(self):
196         """
197         Iteratively removes nodes from solution path to find a shorter path
198         which is still collision-free.
199         Input:
200             None
201         Output:
202             None, but should modify self.path
203         """
204         ########## Code starts here ##########
205 
206         success = False
207 
208         while not success:
209             success = True
210             for i in range(1, len(self.path)-1):
211                 if np.allclose(self.path[i], self.x_goal) or
    np.allclose(self.path[i], self.x_init):
212                     continue
213 
214                 if self.is_free_motion(self.obstacles, self.path[i-1],
    self.path[i+1]):
215                     self.path = np.delete(self.path, i, axis=0)
216                     success = False
217                     break
218 
219         ########## Code ends here ##########
220 
221 class GeometricRRT(RRT):
222     """
223     Represents a geometric planning problem, where the steering solution
224     between two points is a straight line (Euclidean metric)
```

```
    """

    def find_nearest(self, V, x):
        # Consult function specification in parent (RRT) class.
        ########## Code starts here ##########
        # Hint: This should take 1-3 line.
        distances = np.apply_along_axis(lambda i: np.linalg.norm((i - x)), 1, V)
        result = np.argmin(distances)

        return result
        ########## Code ends here ##########
        pass

    def steer_towards(self, x1, x2, eps):
        # Consult function specification in parent (RRT) class.
        ########## Code starts here ##########
        # Hint: This should take 1-4 line.

        diff = x2 - x1
        if np.linalg.norm(diff) < eps:
            return x2

        result = eps * diff / np.linalg.norm(diff) + x1

        return result

        ########## Code ends here ##########
        pass

    def is_free_motion(self, obstacles, x1, x2):
        motion = np.array([x1, x2])
        for line in obstacles:
            if line_line_intersection(motion, line):
                return False
        return True

    def plot_tree(self, V, P, **kwargs):
        plot_line_segments([(V[P[i],:], V[i,:]) for i in range(V.shape[0]) if P[i] >=
0], **kwargs)

    def plot_path(self, **kwargs):
        path = np.array(self.path)
        plt.plot(path[:,0], path[:,1], **kwargs)

```